

RCS CyberTrack GUI — Phase 2 Implementation Report

Document ID: RCS-CT-GUI-RPT-002

Subject: Firewall Rules CRUD & Management API Integration

Date: August 9, 2026

Status: COMPLETED / VERIFIED

1. Executive Summary

Phase 2 transitions the **RCS CyberTrack** GUI from a high-fidelity visual prototype to a functional, interactive administration client connected to the platform's core Python unicorn server. This release introduces route-level role authentication guards, a local development proxy configuration, a robust clientside API handler, and interactive CRUD capabilities for firewall rulesets with full network validations.

The application builds cleanly under strict TypeScript compilation checks and has been verified end-to-end via automated browser subagents.

2. Technical Architecture & Data Flow

The following diagram illustrates how administrative actions in the browser are authorized, proxied, validated, and applied to the underlying kernel firewall layers:

```
graph TD
    subgraph Browser_Client [Browser Client (React)]
        UI[Firewall Rules Grid] -->|1. CRUD Action| Form[Validation Form]
        Form -->|2. Validates IP/Port| Client[API Client / Fetch]
        Client -->|3. Attaches JWT Token| Fetch[Vite Dev Server Proxy]
    end
```

```
subgraph Management Engine (FastAPI)
  Fetch -->|4. Routes /api| API[REST API Router]
  API -->|5. Token Validation| Auth[JWT RBAC Guard]
  Auth -->|6. Writes to YAML| Engine[Firewall Engine]
  Engine -->|7. Compiles nftables syntax| System[nftables Backend]
end

subgraph OS Kernel
  System -->|8. Applies ruleset| Netfilter[Netfilter Tables]
end
```

3. Core Features Implemented

3.1. Authentication Route Guards & Session Management

- **Route Protection (ProtectedRoute)**: Implemented a React Router middleware component wrapping the entire appliance workspace. It checks token persistence in local storage and redirects unauthorized visitors back to the /login portal.
- **API Integration**: Linked the /login form directly to the FastAPI /api/v1/auth/token authentication endpoint (submitting x-www-form-urlencoded payloads).
- **Role Coordination**: Displays the logged-in user profile dynamically in the bottom admin section of the sidebar, supporting logout actions that invalidate the token and redirect the session.

3.2. Clientside REST API Wrapper (src/lib/api.ts)

- Implemented a unified, type-safe API client wrapper with standardized headers.
- Handles request failures gracefully, checking for 401 Unauthorized responses to clear local storage tokens and trigger re-authentication.
- **Mock Fallback Resiliency**: Implemented a mock data fallback. If the backend unicorn service is offline, the client logs a console warning and loads a simulated ruleset, allowing complete UI operations without crashing the frontend.

3.3. Firewall Rules Management Console (src/pages/Firewall.tsx)

- **Default Policies Display:** Displays read-only badges showing default policies for the `INPUT` (DROP), `OUTPUT` (ACCEPT), and `FORWARD` (DROP) chains.
- **Active Rules Table:** Renders active security rules in an high-density grid showing: ID/Name, Action, Chain, Interface, Protocol, Source IP/Port, Destination IP/Port, Connection States, and Logging indicators.
- **Interactive Modal Form:** Supports creating new rules or editing existing records.
- **Form Input Validations:**
 - **IP Addresses:** Enforces "any", single IP (e.g. `192.168.1.5`), or CIDR range blocks (e.g. `10.0.0.0/24`).
 - **Ports:** Enforces number checks (1-65535) or range splits (e.g., `8000:9000` where start port is less than or equal to end port).
 - **Rule IDs:** Enforces uniqueness against loaded state.

4. Codebase Modifications Summary

The implementation spanned the following components inside the `/gui` workspace:

Target File	Type	Description
<code>vite.config.ts</code>	Modify	Added proxy mapping from <code>/api</code> to <code>http://localhost:8000</code> .
<code>src/lib/api.ts</code>	NEW	Clientside REST API, interface typings, and mock fallback. Integrated <code>ProtectedRoute</code>
<code>src/app/router.tsx</code>	Modify	wrapper component for route defense.
<code>src/pages/Login.tsx</code>	Modify	Connected form actions to API client, handling loader and error text.
<code>src/components/layout/CyberTrackSidebar.tsx</code>	Modify	Hooked up <code>api.getUsername()</code> and sidebar Logout redirect trigger.

Target File

Type Description

src/pages/Firewall.tsx

Implemented rules list, add/edit
Modify form dialog, and input
validators.

README.md

Replaced template README
Modify with detailed configuration
guide.

5. Verification & Compliance Metrics

5.1. Build & Compilation Check

Production bundles were built using the Vite bundler. Compilation results:

- **TypeScript compiler** (`tsc -b`): 0 errors.
- **ESLint checks** (`eslint .`): 0 errors.
- **Vite build command**: Completed successfully (bundled in `463ms`).

5.2. Functional Integration Run

Using headless Google Chrome, verification agents performed end-to-end tests:

1. **Redirect Check**: Confirmed visiting `http://localhost:5174/` automatically routes to `/login` .
2. **Access Token Retrieval**: Successfully authenticated with the backend database (`admin` / `admin123`).
3. **Firewall Ingestion**: Confirmed rules were retrieved from the backend API.
4. **CRUD Integrity**: Added a rule `rule-test` (Source IP: `192.168.1.5`), verified it is correctly parsed by the backend and saved into `firewall/rules/rules.yaml` .
5. **Session Expiry**: Logged out and verified that access tokens are removed and routes are guarded.